Daily Quant Digest: 2026-10-26

Today's topics

Track	Topic	Difficulty
Probability & Statistics	Restart, day 5: Markov and Chebyshev, guarantees that need almost no information	Medium
Derivatives	Straddles and strangles: buying a move, on a EURUSD week with an ECB meeting	Medium
Algorithms & DS	Dynamic programming: Russian Doll Envelopes (longest increasing subsequence in 2D)	Medium-Hard
Music	The Doors, "Riders on the Storm" (1971)	

1. Probability & Statistics: Markov and Chebyshev Inequalities

1.1 The one idea: not everyone can be rich

Imagine a room of 1,000 traders. You are told only one fact: their **average** bonus is $10,000. Nobody's bonus is negative. How many of them could have a bonus of $50,000 or more?

Think of the total money as a fixed pile: $1{,}000 \times \$10{,}000 = \$10$ million. Every trader with $50,000 or more uses up at least $50,000 of that pile. So at most $\$10\text{M}/\$50{,}000 = 200$ traders can be that lucky. If 201 were, the pile would already be overspent, even if everybody else got zero.

That is the whole of **Markov's inequality**. For any $X \geq 0$ and any $a > 0$:

$$P(X \geq a) \leq \frac{E[X]}{a}.$$

Here $10{,}000/50{,}000 = 0.2$, which is 200 out of 1,000. Notice what we did *not* need: the shape of the distribution, the variance, independence. Only the mean and the fact that nobody goes below zero. That is why inequalities matter in quant work: when you do not trust your model, they give you a bound that holds **for every possible model**.

The price you pay is that the bound is often loose. But it cannot be improved in general. The extreme case is “200 traders get exactly $50,000, 800 get nothing”. That room has mean $10,000 and exactly 20% at $50,000. So Markov is tight: some distribution hits it exactly.

1.2 Chebyshev: Markov wearing glasses

Markov only uses the mean. Usually we also know the spread. Chebyshev’s trick is simple: apply Markov not to X, but to the **squared distance from the mean**, $(X-\mu)^2$. That is always ≥ 0, and its average is the variance σ^2. So:

$$P(|X-\mu| \geq k\sigma) = P\big((X-\mu)^2 \geq k^2\sigma^2\big) \leq \frac{\sigma^2}{k^2\sigma^2} = \frac{1}{k^2}.$$

In words: **at most $1/k^2$ of any population can sit k standard deviations or more away from the average.**

Back to 1,000 traders, now looking at their daily P&L. Whatever the distribution:

Distance from mean	Chebyshev: at most	If P&L were normal
2 standard deviations	250 traders	about 46
3 standard deviations	111 traders	about 3

The gap between the two columns is the point. The normal distribution is a very *optimistic* assumption about tails. Chebyshev tells you how bad things could be if the tails are as fat as mathematically possible for that variance. Real P&L lives somewhere in between, and often closer to Chebyshev than people like.

1.3 The one-sided version (Cantelli)

Risk managers usually care about one side only: big losses, not big gains. Chebyshev counts both tails, so for one tail it wastes some of its budget. The sharper one-sided version is **Cantelli’s inequality**: for $a > 0$,

$$P(X-\mu \geq a) \leq \frac{\sigma^2}{\sigma^2 + a^2}.$$

Picture a seesaw balanced at the mean. To put weight far out on the right, you must put weight on the left to keep the balance (the mean), but the variance limits how far out anything can sit. The worst case uses exactly two values, one at $\mu + a$ and one a little below the mean.

1.4 Common mistakes

- **Using Markov on something that can be negative.** P&L, returns and changes in price can be negative. The “pile of money” argument breaks: losers can refund the winners. Apply Markov to $|X|$, X^2, or e^{tX} instead (the last one leads to the Chernoff

bound, for another day).
- **Forgetting the bound can be useless.** For $k \le 1$, Chebyshev says the probability is at most 1 or more. True, but empty. Inequalities are informative only far enough into the tail.

1.5 Interview question

A random variable $X \ge 0$ has mean $E[X] = 10$.

a. What is the largest possible value of $P(X \ge 50)$? Give a distribution that achieves it.

b. Now you are also told that the standard deviation is 10. What is the best upper bound on $P(X \ge 50)$ now? Is it achieved by a non-negative X?

c. You average n independent copies of a variable with standard deviation 10. How large must n be to **guarantee** that the sample average is within 1 of the true mean with probability at least 95%? Compare with what the normal approximation would suggest.

1.6 Solution

(a) Markov: $P(X \ge 50) \le 10/50 = 0.2$. It is achieved by "$X = 50$ with probability 0.2, $X = 0$ otherwise". Mean: $0.2 \times 50 = 10$. Check. Answer: **0.2**.

(b) Now the variance is 100. Use Cantelli with $a = 50 - 10 = 40$:

$$P(X \ge 50) \le \frac{100}{100 + 1600} = \frac{1}{17} \approx 0.059.$$

(Plain two-sided Chebyshev gives $100/1600 = 1/16$, slightly worse.)

Is it achievable? Try two values: 50 with probability $p = 1/17$, and y with probability $16/17$. The mean must be 10: $50/17 + 16y/17 = 10$ gives $16y = 120$, so $y = 7.5$. That is non-negative, so allowed. Check the variance: $E[X^2] = (2500 + 16 \times 56.25)/17 = (2500 + 900)/17 = 200$, so $\mathrm{Var} = 200 - 100 = 100$. Check. Answer: $1/17$**, and it is tight.**

Compare: if X were normal with mean 10 and sd 10, then $P(X \ge 50) = P(Z \ge 4) \approx 0.00003$. The fat-tailed worst case is about 2,000 times larger. This is why "a 4-sigma day should happen once in 100 years" is a statement about the normal model, not about markets.

(c) The sample average $\bar{X}$ has standard deviation $10/\sqrt{n}$, so variance $100/n$. Chebyshev:

$$P(|\bar{X} - \mu| \ge 1) \le \frac{100/n}{1^2} = \frac{100}{n}.$$

We need $100/n \le 0.05$, so $n \ge 2{,}000$.

The normal approximation (CLT) instead asks $1.96 \times 10/\sqrt{n} \le 1$, giving $n \ge 384.2$, so $n = 385$.

Interview trick. Say the answer in one line: "Chebyshev's 95% band is $\sqrt{20} \approx 4.47$ standard errors wide, against 1.96 for the normal, so I need about $(4.47/1.96)^2 \approx 5.2$ times more samples." Indeed $2000/385 \approx 5.2$.

Sanity check. The guarantee must cost more than the approximation, because it has to hold even for the nastiest distribution with sd 10. It does: 2,000 against 385. And as n grows, both shrink the error like $1/\sqrt{n}$, which is the law of large numbers in both cases.

---

2. Derivatives: Straddles and Strangles

2.1 The setup: who trades this and why

It is Monday. On Thursday the **European Central Bank** announces its rate decision. A macro hedge fund is sure the meeting will move EURUSD a lot, but it honestly does not know which way. A surprise cut sends the euro down. A hawkish surprise sends it up. A "boring" meeting would leave it where it is.

The fund wants a position that **pays if the market moves far, in either direction**. That is a **straddle**: buy a call and a put with the same strike and expiry.

On the other side is a bank's FX options desk, or a fund that likes selling options. They think the market is over-excited and the move will be smaller than people fear. They sell the straddle and collect the premium, like an insurer who believes the storm will miss.

2.2 The straddle: a bet on distance, not direction

Picture a stake in the ground at the strike, and a rope tied to it. At expiry, the straddle pays you **the length of the rope**, that is, the distance between spot and the strike, whichever side spot ends up on:

$$\text{Payoff} = |S_T - K|.$$

The call pays if spot ends above K, the put pays if it ends below. Only one of them can pay. The payoff chart is a **V**.

Because the call has a delta of about $+0.5$ and the put about -0.5, the at-the-money straddle has a delta of about zero. You have no view on direction. (In FX, this is why the market quotes the "ATM" strike as the **delta-neutral straddle** strike, where the two deltas cancel exactly.)

You win if the rope at expiry is longer than the premium you paid. So the breakevens are $K \pm \text{premium}$.

2.3 The deep idea: the straddle price is the market's expected move

Here is the part worth remembering. For a short-dated at-the-money straddle,

$$\text{Straddle} \approx \sqrt{\tfrac{2}{\pi}}\,\sigma\sqrt{T}\,S \approx 0.8\,\sigma\sqrt{T}\,S.$$

Why? $\sigma\sqrt{T}\,S$ is one standard deviation of the move until expiry. The straddle pays the **absolute** size of the move, and for a normal variable the average absolute value is $\sqrt{2/\pi} \approx 0.8$ standard deviations. So:

> The price of an ATM straddle **is** the market's estimate of the average size of the move.

That turns the trade into a simple question: **will the actual move be bigger or smaller than the market's priced move?** Buying a straddle is buying realized volatility at the price of implied volatility.

2.4 The strangle: the cheaper, lazier cousin

A **strangle** buys an out-of-the-money put and an out-of-the-money call, for example the 25-delta put and the 25-delta call. Now there is a flat zone in the middle where nothing pays. The payoff chart is a **bathtub** instead of a V.

The analogy is car insurance with a deductible. A straddle pays from the first euro of movement. A strangle pays nothing for small moves, but costs much less. You buy a strangle when you think the move will be **really** large, or when you want to sell premium but keep a cushion.

2.5 The worked trade

Market on Monday:

- EURUSD spot 1.1600. One-week implied vol **9%** (higher than usual because the week contains the ECB meeting).
- USD rate 4%, EUR rate 2%, $T = 7/365$.
- Notional **EUR 10 million**. Then 1 pip (0.0001) of payoff is worth **USD 1,000**.

Prices from Garman-Kohlhagen:

Structure	Strikes	Premium (pips)	Premium (USD)	Breakevens at expiry
ATM straddle	1.1600 call + put	115.3	115,300	1.1485 / 1.1715
Strangle (about 25-delta)	1.1500 put + 1.1700 call	41.9	41,900	1.1458 / 1.1742

Check the rule of thumb: one standard deviation of the weekly move is $0.09 \times \sqrt{7/365} \times 1.16 \approx 0.0145$, i.e. 145 pips. Times 0.8 gives about 115 pips. Exactly the straddle price. The market is saying: "we expect EURUSD to move about 115 pips this week, on average."

Now four endings at expiry (ignoring the tiny one-week discounting):

Spot at expiry	Straddle P&L	Strangle P&L
1.1600 (nothing happens)	$-115{,}300$	$-41{,}900$
1.1550 (small move)	$50 - 115.3 = -65.3$ pips: $-65{,}300$	$-41{,}900$
1.1400 (dovish surprise)	$200 - 115.3$: $+84{,}700$	$100 - 41.9$: $+58{,}100$
1.1200 (shock)	$400 - 115.3$: $+284{,}700$	$300 - 41.9$: $+258{,}100$

Two lessons. First, the straddle wins whenever the move beats 115 pips, either way. Second, the strangle loses less when nothing happens and, **per dollar of premium**, wins far more in a shock (about 6 times its premium, against about 2.5 for the straddle). It is the lottery ticket version.

After the meeting the event is gone, and one-week vol typically drops back, for example from 9% to 6%. This is the **vol crush**. If the fund still holds options after Thursday, the remaining days are worth much less. Event buyers usually need the move to come on the day, not later.

2.6 Greeks in plain words

- **Long straddle or strangle:** long gamma (you like moves), long vega (you like vol going up), short theta (you pay rent every day).
- **Short straddle or strangle:** the mirror image. You earn the rent daily, and you are exposed to the one big day that wipes it out.

2.7 When it went wrong: OptionSellers.com, November 2018

A US fund manager ran a strategy of selling far out-of-the-money options on natural gas futures, a “strangle-style” premium collection that had earned steady income for years. In mid-November 2018, natural gas prices jumped very sharply in a few days. The short calls exploded in value, client accounts were wiped out, and many clients ended up owing money to their broker on top. The lesson: selling strangles is selling insurance. The premium looks like free money right up to the storm.

2.8 Likely interview questions

1. **Why is an ATM straddle worth about** $0.8\,\sigma\sqrt{T}\,S$**?** It pays $|S_T - K|$. With $K \approx S$, that is the absolute move, whose mean under a normal is $\sqrt{2/\pi}$ times its standard deviation $\sigma\sqrt{T}S$.
2. **You are long a straddle and spot does not move, but implied vol jumps. P&L?** Positive: the straddle is long vega. You can win on a straddle without any move, through vol, before expiry.
3. **Straddle or strangle for an event you think the market is underpricing?** If you think the move will be large but not extreme, the straddle (pays from the first pip). If you think it is a tail event, the strangle gives far more payoff per dollar.

---

3. Algorithms & Data Structures: Russian Doll Envelopes

3.1 Problem

You have n envelopes, each with a width w and a height h. One envelope fits inside another if and only if it is **strictly** smaller in **both** width and height. You cannot rotate envelopes.

Return the maximum number of envelopes you can nest inside each other, like Russian dolls.

Constraints: $1 \le n \le 10^5$, $1 \le w, h \le 10^5$.

Example 1. Input `[[5,4],[6,4],[6,7],[2,3]]`. Output **3**: $[2,3] \to [5,4] \to [6,7]$.

Example 2. Input `[[1,1],[1,1],[1,1]]`. Output **1**: equal envelopes do not nest.

Example 3. Input `[[1,1],[2,2],[2,3],[2,4]]`. Output **2**. The three envelopes of width 2 cannot nest in each other, because their widths are equal. This example is the trap.

3.2 From first principles

Step 1: kill one dimension by sorting. Two dimensions are hard to think about. If we sort the envelopes by width, then any chain must go left to right in the sorted list. Now the question looks like: "pick a subsequence where the heights strictly increase". That is the classic **longest increasing subsequence** (LIS).

Step 2: the brute force DP. Let $\text{best}[i]$ be the longest chain that ends with envelope i. Then

$$\text{best}[i] = 1 + \max\{\text{best}[j] : j < i,\ w_j < w_i,\ h_j < h_i\}.$$

This is correct and simple, but it compares every pair: $O(n^2)$. With $n = 10^5$ that is 10^{10} steps, far too slow. We keep it as a checker.

Step 3: the equal-width trap. If we sort by width and then just take the LIS of heights, example 3 goes wrong. Sorted as $[1,1], [2,2], [2,3], [2,4]$, the heights $1, 2, 3, 4$ strictly increase, so LIS says 4. But three of those envelopes have the same width.

The fix is a beautiful trick: **for equal widths, sort heights in decreasing order.** Then the heights read $1, 4, 3, 2$. Within one width group the heights go down, so an increasing subsequence can take **at most one** envelope from each group. That is exactly the rule "equal widths cannot nest". The LIS is now 2. Correct.

Step 4: LIS in $O(n \log n)$, the patience idea. Picture dealing cards into piles in a game of patience. Keep an array `tails`, where `tails[k]` is the **smallest possible last height** of an increasing chain of length $k+1$ seen so far.

Why the smallest? A chain that ends low is more useful: more future envelopes can extend it. It is like keeping, for each team size, the team whose tallest member is shortest.

For each new height h:

- If h is bigger than every tail, it extends the longest chain: append it.
- Otherwise find the first tail that is $\geq h$, and replace it with h. We found a cheaper ending for a chain of that length.

`tails` always stays sorted, so "find the first tail $\geq h$" is a binary search. The answer is the final length of `tails`.

Walk example 1. Sorted (width up, height down for ties): $[2,3], [5,4], [6,7], [6,4]$. Heights: $3, 4, 7, 4$.

Height	tails before	Action	tails after
3	[]	append	[3]
4	[3]	append	[3, 4]
7	[3, 4]	append	[3, 4, 7]
4	[3, 4, 7]	first tail ≥ 4 is 4: replace	[3, 4, 7]

Length 3. Note that `tails` is **not** necessarily an actual chain; only its length is guaranteed to be right.

Why strict uses `lower_bound`. Heights must strictly increase, so an equal height cannot extend a chain. `lower_bound` finds the first tail $\geq h$ and overwrites it, so a tie never grows the array. (`upper_bound` would be for non-decreasing chains.)

Complexity. Sorting is $O(n \log n)$. Each of the n binary searches is $O(\log n)$. Total $O(n \log n)$ **time**, $O(n)$ **extra space**.

3.3 C++ solution

```cpp
#include <algorithm>
#include <cassert>
#include <iostream>
#include <random>
#include <vector>

using namespace std;

// Sort by width ascending; for equal widths, by height DESCENDING.
// Then the answer is the longest strictly increasing subsequence of heights.
// tails[k] = smallest possible last height of an increasing chain of length k+1.
int maxEnvelopes(vector<vector<int>> envelopes) {      // by value: we sort our own copy
    sort(envelopes.begin(), envelopes.end(),
         [](const vector<int>& a, const vector<int>& b) {
             if (a[0] != b[0]) return a[0] < b[0];      // narrower first
             return a[1] > b[1];                         // same width: taller first
         });

    vector<int> tails;
```

```
    tails.reserve(envelopes.size());
    for (const auto& e : envelopes) {
        int h = e[1];
        // first tail that is >= h (strict increase, so equal heights cannot extend)
        auto it = lower_bound(tails.begin(), tails.end(), h);
        if (it == tails.end()) tails.push_back(h);      // h extends the longest chain
        else *it = h;                                    // cheaper ending for that length
    }
    return static_cast<int>(tails.size());
}

// Brute force O(n^2) DP for cross-checking.
int bruteForce(vector<vector<int>> env) {
    sort(env.begin(), env.end());
    int n = static_cast<int>(env.size()), ans = 0;
    vector<int> best(n, 1);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j)
            if (env[j][0] < env[i][0] && env[j][1] < env[i][1])
                best[i] = max(best[i], best[j] + 1);
        ans = max(ans, best[i]);
    }
    return ans;
}

int main() {
    vector<vector<int>> ex1 = {{5, 4}, {6, 4}, {6, 7}, {2, 3}};
    vector<vector<int>> ex2 = {{1, 1}, {1, 1}, {1, 1}};
    vector<vector<int>> ex3 = {{1, 1}, {2, 2}, {2, 3}, {2, 4}};
    assert(maxEnvelopes(ex1) == 3);
    assert(maxEnvelopes(ex2) == 1);
    assert(maxEnvelopes(ex3) == 2);

    mt19937 rng(20261026);
    for (int trial = 0; trial < 3000; ++trial) {
        int n = static_cast<int>(rng() % 40) + 1;
        int range = static_cast<int>(rng() % 10) + 1;    // small range forces many ties
        vector<vector<int>> env(n);
        for (auto& e : env)
            e = {static_cast<int>(rng() % range) + 1, static_cast<int>(rng() % range) + 1};
        assert(maxEnvelopes(env) == bruteForce(env));
    }
    cout << "all tests passed\n";
    return 0;
}
```

The full file `2026-10-26.cpp` also prints the examples and runs a 100,000-envelope test. It passes all three examples and 3,000 random cross-checks against the brute force, and the large test runs in a few milliseconds.

3.4 Walkthrough of the C++

- **Pass by value on purpose.** `maxEnvelopes(vector<vector<int>> envelopes)` takes a copy, because we are going to sort it and we should not reorder the caller's data. If the caller does not need their vector afterwards, they can write `maxEnvelopes(std::move(v))` and no copy happens at all: the vector's buffer is simply handed over.

- **The lambda comparator.** `[](const vector<int>& a, const vector<int>& b) { ... }` is an unnamed function written in place. The empty `[]` means it captures nothing from the surrounding code. The parameters are `const` references, so `sort` does not copy two small vectors on every comparison (that would be slow and allocate memory). The comparator must be a **strict weak ordering**: it must return `false` for equal elements. Using `<` and `>` (never `<=`) guarantees that. A comparator with `<=` is undefined behaviour and can crash `std::sort`.
- **`reserve`.** `tails.reserve(envelopes.size())` allocates the maximum memory once, so `push_back` never has to grow and copy the buffer. It changes capacity, not size: `tails` is still empty.
- **Range-for with `const auto&`.** `for (const auto& e : envelopes)` visits each envelope by reference, without copying the inner vector. `auto` lets the compiler work out the type (`vector<int>`).
- **`lower_bound` and iterators.** `lower_bound(first, last, h)` binary-searches a **sorted** range and returns an **iterator** to the first element that is not less than h, or `last` (here `tails.end()`) if there is none. An iterator is a generalised pointer: `*it = h` writes through it, directly into the vector. Using `lower_bound` on an unsorted range gives wrong answers silently, so the invariant "tails is sorted" is what makes the algorithm legal.
- **`static_cast<int>`.** `size()` returns `size_t`, an unsigned type. The explicit cast says "I know this fits in an int", and keeps `-Wall` quiet about signed/unsigned mixing.
- **The tester.** `mt19937` is the Mersenne Twister random generator. A fixed seed makes failures reproducible. Small coordinate ranges create lots of equal widths and heights, which is exactly where the tie-breaking bug would hide. `assert` stops the program if any answer disagrees with the brute force.
- **Brute force sorting.** In `bruteForce`, `sort(env.begin(), env.end())` uses `vector`'s built-in `<`, which compares lexicographically: by width, then height. That is enough, because the brute force checks both strict conditions explicitly.

Where this pattern shows up next: box stacking, "longest chain of pairs", scheduling jobs with two constraints, and any problem where sorting one coordinate turns a 2D question into a 1D LIS.

---

4. Music: The Doors, "Riders on the Storm" (1971)

Context

"Riders on the Storm" is the closing track of **L.A. Woman**, the sixth studio album by **The Doors**, released in April 1971. Like most Doors songs, it is credited to the whole band: Jim Morrison (vocals), Ray Manzarek (keyboards), Robby Krieger (guitar) and John Densmore (drums). On the album, the bass was played by session musician Jerry Scheff, who also played with Elvis Presley.

The story behind it

L.A. Woman was recorded in the band's own rehearsal space in Los Angeles, with engineer Bruce Botnick co-producing alongside the band, after their long-time producer Paul Rothchild stepped away from the project. The relaxed, live-in-a-room setting gave the album a looser, bluesier feel than their earlier records.

The song grew out of the band playing around with the old cowboy song "(Ghost) Riders in the Sky", slowed down into a dark, smoky groove. It became the last song The Doors recorded with Morrison. He moved to Paris shortly after the sessions, and he died there in July 1971, only weeks after the single came out. That timing has made the song feel like a farewell ever since.

Why people love it

It is pure atmosphere. Most rock songs push forward; this one hangs in the air like a wet night on an empty highway. It is slow, quiet and slightly menacing, and it sounds like nothing else from the time. Many fans consider it the most perfect late Doors recording: a band at ease, playing with restraint.

What to listen for

- **The rain and thunder.** The track opens and closes with recorded sounds of a storm. They are not decoration; they set the whole mood before a note is played.
- **Manzarek's electric piano.** He plays a Fender Rhodes, with its soft, bell-like tone. His falling runs sound like raindrops. This is the instrument that carries the song.
- **The whisper.** Morrison sings the main vocal, and a second, whispered vocal track follows him like an echo or a ghost. Listen with headphones.
- **The bass line.** A simple, steady, repeating figure underneath everything. It is the road the storm is moving over.
- **Restraint.** Krieger's guitar and Densmore's drums stay soft and jazzy. Nobody shows off. Notice how much tension comes from playing quietly.

Play it late at night, ideally on headphones, and let the seven minutes go by without skipping.